일은 AI가 하고
돈은 제가 받습니다

윤준혁

TSBM

저는 이런 사람입니다

- 당근 운영개발팀에서 **KAMP**라는 제품을 개발하고 있어요
- KAMP는 **Karrot Agent Management Platform**의 약자로, No-code/low-code로 AI 에이전트를 구성할 수 있는 플랫폼 제품이에요

오늘 할 이야기

01
에이전트

02
도구 &
멀티 에이전트

03
지식 관리 &
최적화

04
모니터링 & 평가

01

에이전트

사람이 처리하기엔 너무 많은 일

일간

문의

3,000건

신고

100,000건

챗봇

1

어드민에서
운영자를 도와주는
형태

2

질문에 답변은 잘 함
실제 시스템은 못
건드림

3

결국 처리는
사람이 해야 하는
상태

→ 효과는 미미했다!

워크플로우

1

정해진 분기를
따라가는 형태

2

새로운 케이스마다
분기 추가

3

현실의 복잡한
판단을
커버 못함

→ 유지보수 지옥

에이전트

챗봇

질문 → 답변
대화만

❌ 처리 불가

워크플로우

트리거 → 정해진 액션
고정 분기

❌ 유연성 부족

에이전트

목표 → 스스로 판단 →
도구 선택 → 실행
자율성

✅ 자율 판단

운영엔 에이전트가 딱이야

정책 읽기 → 맥락 이해 → 판단 → 실행

핵심: 자율성 — 목표를 주면 스스로 계획하고 실행

02

도구 & 멀티 에이전트

도구: 에이전트의 손발

- 도구 없는 에이전트 = 생각만 하는 존재
- 도구를 통해 현실 세계와 연결
- 에이전트가 스스로 어떤 도구를 쓸지 판단

도구 설계가 에이전트 품질의 절반

운영 자동화에 사용했던 도구들

콘텐츠 조회
처리 대상 내용 읽기

정책 조회
판단 기준 가져오기

처리 API 호출
실제 액션 실행

알림 생성
사용자에게 안내

도구에 대한 설명의 중요성

01

Description을 명확하게

언제/어떻게 써야 하는지
설명이 핵심

02

파라미터 스키마 정확하게

타입, 필수 여부, 허용 값 등을 정확히 정의

03

일부 파라미터는 고정값이 더 안전

LLM이 매번 생성하는 것보다
고정값이 나은 경우 있음

에이전트 하나로 다 하면?

컨텍스트 과부하

→ 성능 저하

역할 경계 모호

→ 예측 불가 동작

오류 추적 어려움

→ 어디서 틀렸는지 모름

→ 역할 분리 필요

에이전트 팀

분류

- 콘텐츠 조회
- 처리 항목 선택

검토

- 정책 조회
- 수락/거부 판단
- (주관 배제)

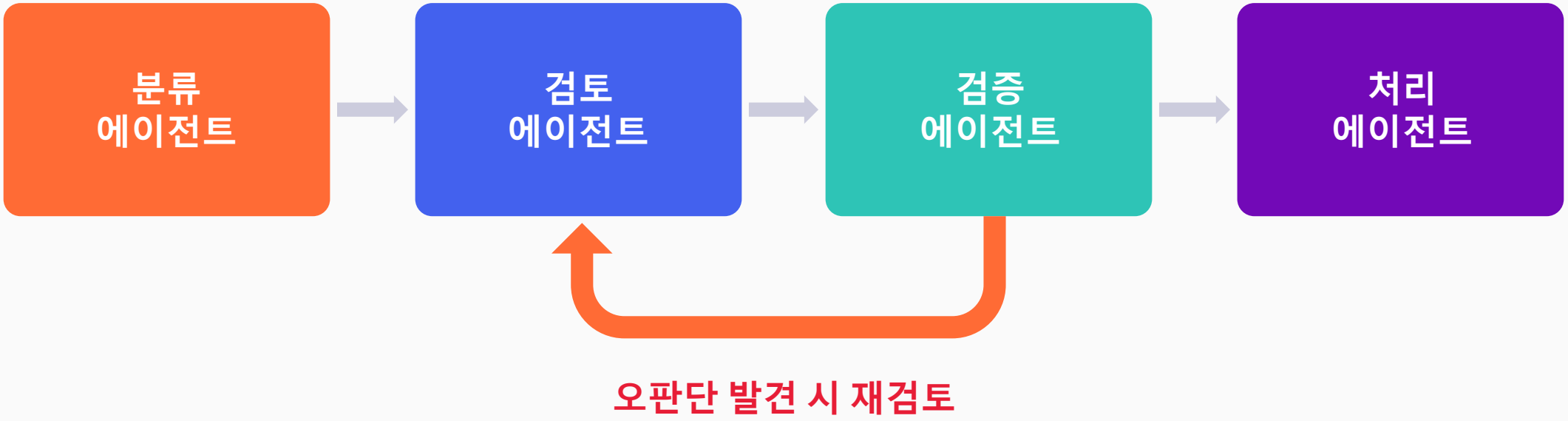
검증

- 체크리스트 실행
- 오판단 발견 시
- 재검토 루프

처리

- 우선순위 선택
- 알림 생성
- API 호출

에이전트 팀의 협업 과정



Self-correction의 효과

- 검증 에이전트(Critic)가 오판단을 잡아냄
- 체크리스트:
  - ✓ 명확한 근거가 있는가?
  - ✓ 정책 조건을 정확히 적용했는가?
  - ✓ 근거와 결론이 일치하는가?
  - ✓ 추정 기반 판단 금지 ("~로 보인다", "~일 가능성이 있다")

→ 정확도 개선에 큰 역할

03

지식 관리 & 최적화

지식 관리는 프롬프트 밖에서

기술적 이유

- 컨텍스트 비대, lost-in-the-middle 성능 저하
- 토큰 비용 폭증
- 정책 변경 시 프롬프트 수정 필요

조직적 이유

- 비엔지니어도 에이전트를 개선할 수 있어야 함
- 정책 담당자: 문서만 수정 → 에이전트 판단 변경
- 프롬프트 = 엔지니어 / 정책 = 담당자

→ 오너십 분리 → 빠른 개선 사이클

RAG = 꼭 벡터 검색이어야 할까?

RAG 검색 방식

- 벡터 검색
- 키워드 검색
- API 조회
- DB 쿼리

KAMP 사례

ID로 정책 API 직접 조회
→ 벡터 DB 없이 충분

데이터 구조에 맞는 방법이 최선 — 오버엔지니어링 주의

에이전트가 지식 개선에도 참여

- 잘못된 지식이 잘못된 동작의 원인이 되는 경우가 많음
- 사람이 인지하기 쉽지 않음
- 에이전트에게 로깅 도구를 제공하고 모호한/모순이 있는 내용 발견 시 직접 피드백 기록
- 담당자 확인 → 정책 보완 → 에이전트 재판단

- **에이전트 = 단순 실행자가 아닌 시스템 개선 참여자**

모델 선택: 모두 같은 모델?

에이전트마다
요구사항이 다름

반복 판단 vs
정밀 검증은 다름

비용/성능/레이턴시
고려 필요

비용/성능/레이턴시 트레이드오프 → 역할별 모델 분리

역할에 따른 모델 선택

결국 직접 써보는 게 제일 좋음 (A/B 테스트)

단순 분류 / 처리 / 도구 실행

→ Flash, Haiku, mini 등
가벼운 모델로 충분

정책 판단 / 논리적 근거 필요

→ Pro, Opus(Sonnet), o3 등
고성능 모델 추천

큰 모델은 레이턴시 무관한 경우에만 사용 추천

비용 모니터링과 가드레일

- 자율적 에이전트는 루프 등으로 비용 비정상 증가 위험

기술적 가드레일:

  - Invocation 횟수 제한, Timeout 설정

모니터링:

  - 에이전트별 비용 추적, 이상 탐지

가드레일과 모니터링 모두 필수

04

평가 & 모니터링

정형화된 응답을 내는 경우

- 이 경우는 쉬움
- 맞았는지/틀렸는지 검사하면 됨
- 중요한 것은 평가 데이터셋의 품질

비결정적 시스템이기 때문에

자율적 에이전트는 루프 등으로 비용 비정상 증가 위험

기술적 가드레일

- Invocation 횟수 제한
- Timeout 설정

모니터링

- 에이전트별 비용 추적
- 이상 탐지

가드레일과 모니터링 모두 필수

비정형 응답을 내는 경우

- LLM-as-a-judge
- 에이전트가 답변이 기준에 맞는지 검토
  - 정책 준수 여부
  - 판단 근거 명확성
  - 근거와 결론 일치
- 에이전트를 구성하는 모델과 다른, 고성능 모델 사용을 권장

모니터링: 모든 것을 남겨라

- 각 에이전트 단계별 로깅 필수
- 오판단 발생 시 어느 에이전트에서 잘못됐는지 추적
- 사용된 모델, 프롬프트, 응답, 비용, 레이턴시 등 모든 것을 남겨야 재현할 수 있음
- 분석도 에이전트가

블랙박스 에이전트는 프로덕션에서 위험

통계 대시보드

- 자주, 쉽게 볼 수 있는 곳에 만들어두면 좋음
- 보면 좋은 지표
  - 호출량
  - 호출 당 비용(평균, 상위 n%)
  - 호출 당 평균 레이턴시
  - 응답 실패율

마무리

정리하면

01
도구를 줘야 진짜 일을 한다

- 챗봇 → 에이전트 전환 핵심
- 시스템 연동 도구 = 실질적 가치
- 답변만으론 효과 미미

02
도구 설계 = 에이전트 성능

- Description을 명확히 작성
- 파라미터 스키마를 정확히
- 고정값으로 안전하게 제어

03
역할을 나눠라

- 멀티 에이전트로 역할 분리
- 컨텍스트 과부하 방지
- Self-correction이 정확도 핵심

정리하면

04
오버엔지니어링을 경계하라

- RAG ≠ 벡터 검색
- 역할에 맞는 모델 선택
- 단순 작업엔 가벼운 모델로 충분

05
가드레일 + 모니터링은 필수

- 비용 제어와 호출 제한 설정
- 단계별 로깅으로 추적 가능하게
- 블랙박스 에이전트는 프로덕션에서 위험

그래서 성과는?

대다수의 중고거래 거래금지물품 신고가 에이전트에 의해 처리

점진적으로 확장할 계획

잘 만든 에이전트 = 팀 전체의 레버리지

감사합니다

윤준혁

junhyeok.dev@gmail.com

X @junhyeok_dev

네트워킹 세션에서 더 이야기해요 🙌